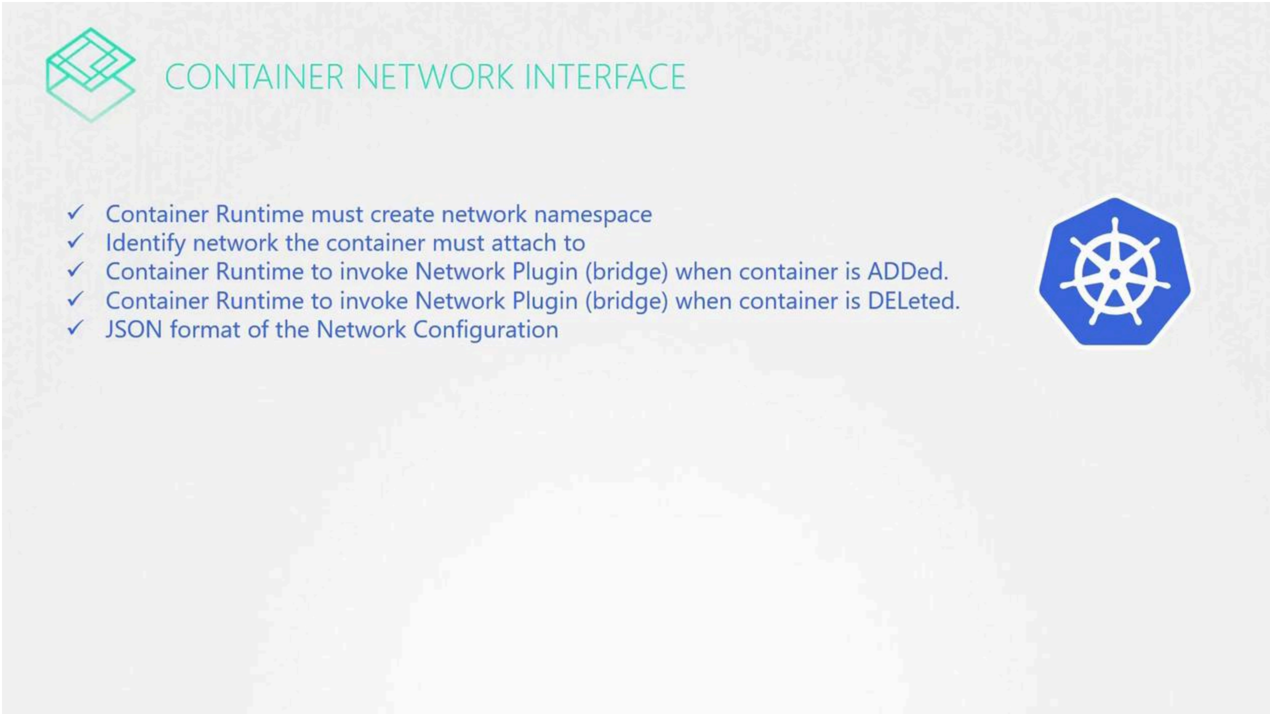
Kubernetes and Cloud Native Associate - KCNA

Container Orchestration Networking

CNI in Kubernetes

Welcome to this lesson on how Kubernetes uses the Container Network Interface (CNI) to configure network plugins for containers. In earlier lessons, we covered the fundamentals of network namespaces, Docker networking, and the emergence of CNI along with its plugins.

In this article, you'll learn how Kubernetes is configured to utilize these network plugins. CNI defines the responsibilities for container runtimes, and in this context, Kubernetes creates container network namespaces and links them to the appropriate network plugins. A dedicated component within Kubernetes first creates the containers and then invokes the specified CNI plugin based on the configuration.



Kubelet Configuration for CNI

The kubelet service on each node is the key component for configuring the CNI plugin. Within the kubelet service file, the network plugin is set to CNI and options are provided that specify the directories for both CNI plugins and configuration files. Here is an example snippet from a kubelet service file:

```
ExecStart=/usr/local/bin/kubelet \
    --config=/var/lib/kubelet/kubelet-config.yaml \
    --container-runtime=remote \
    --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \
    --image-pull-progress-deadline=2m \
    --kubeconfig=/var/lib/kubelet/kubeconfig \
    --network-plugin=cni \
    --cni-bin-dir=/opt/cni/bin \
    --cni-conf-dir=/etc/cni/net.d \
    --register-node=true \
    --v=2
```

When you inspect the running kubelet process with a command like:

```
ps -aux | grep kubelet
```

You will see that the network plugin is set to CNI along with additional options, such as:

- The CNI binaries directory (`` `/opt/cni/bin` ``), which contains executables for supported plugins (e.g., bridge, DHCP, flannel).
- The CNI configuration directory (`` `/etc/cni/net.d` ``), where the kubelet reads configuration files to determine which plugin to use.

For example, you can list the contents of these directories with the following commands:

```
# Display the running kubelet process showing CNI options
ps -aux | grep kubelet

# List CNI plugin executables
ls /opt/cni/bin

# List CNI configuration files
ls /etc/cni/net.d
```

Note

If multiple configuration files exist in the directory, kubelet selects the first file in alphabetical order. A common configuration file is `` `10-bridge.conf` ``.

Example Bridge Configuration

Below is an example bridge configuration file (`10-bridge.conf`) that complies with the CNI standard:

```
{
  "cniVersion": "0.2.0",
  "name": "mynet",
  "type": "bridge",
  "bridge": "cni0",
  "isGateway": true,
  "ipMasq": true,
  "ipam": {
    "type": "host-local",
    "subnet": "10.22.0.0/16",
    "routes": [
      { "dst": "0.0.0.0/0" }
    ]
  }
}
```

To view its content, run:

```
cat /etc/cni/net.d/10-bridge.conf
```

This configuration file includes several key components:

- The `"bridge"` parameter specifies the name of the bridge interface (in this case, `cni0`).
- The `"isGateway"` option determines whether the bridge network should have an IP address to function as a gateway.
- The `"ipMasq"` option configures IP masquerading (NAT rules) for outgoing traffic.
- The `"ipam"` section sets up IP Address Management. Here, `"host-local"` management is used where the `"subnet"` field defines the IP range for the pods (e.g., `10.22.0.0/16`), and the `"routes"` array sets the default routing.

Warning

Be cautious when modifying the IPAM settings, as incorrect configuration can lead to network conflicts or unreachable pods.

Alternatively, you can switch the IPAM type to `"dhcp"` if you prefer to use an external DHCP server for IP address allocation.

That concludes our lesson on how Kubernetes integrates with CNI to manage container networking. For more detailed information, consider exploring **Kubernetes Documentation**.

Watch Video

Watch video content

Previous

Pod Networking

Next

CNI weave